Clinique
Les Baobabs
Soins de qualité · Dakar, Sénégal

# Dossier de conception

Application MédiRDV - Clinique Les Baobabs

Appel d'offres AO-CLB-2026-01 - Livrable L1
Maquettes, modèle de données, architecture, plan de tests

# 1. Contexte et périmètre

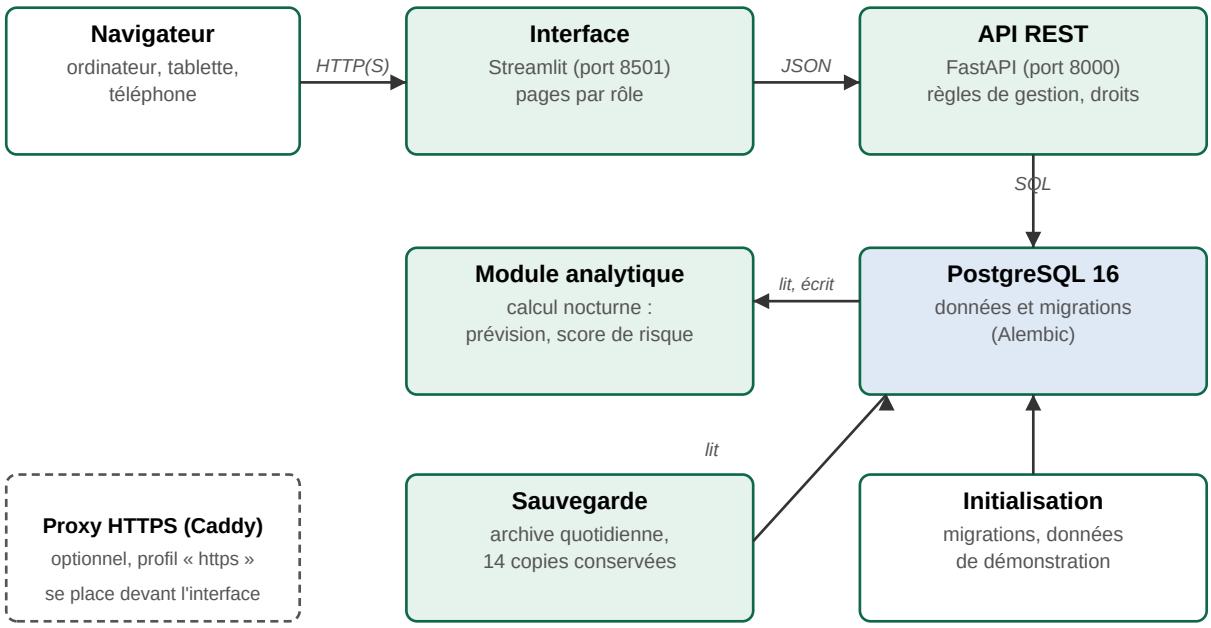
La Clinique Les Baobabs (Dakar) souhaite remplacer la prise de rendez-vous par téléphone et cahier par une application unique. MédiRDV couvre les référentiels (spécialités, médecins, patients), la planification des rendez-vous, un tableau de bord avec prévision et score d'absence, et l'administration (comptes, journal). Le patient n'accède pas directement à l'application dans cette phase.

Les objectifs mesurables retenus : réduire les absences non anticipées grâce à une liste de rendez-vous à relancer, lisser la saturation du lundi et du samedi matin grâce à la carte de chaleur de la demande, et donner à la direction une vue mensuelle fiable.

Ce dossier décrit l'architecture, le modèle de données, les règles de gestion, la sécurité, les maquettes (captures de l'application réelle) et la stratégie de tests, puis la conformité aux exigences du cahier des charges.

# 2. Architecture

Quatre briques Docker Compose, lancées par une seule commande : base de données, API, interface et module analytique, avec un service de sauvegarde et un proxy HTTPS optionnel.




| Brique | Technologie | Rôle |
|---|---|---|
| Interface | Streamlit | Pages par rôle en français ; aucune règle métier côté interface (tout est revérifié par l'API). |
| API | FastAPI, Pydantic v2, SQLAlchemy 2 | Règles de gestion RG-01 à RG-11, droits par rôle, journal ; documentation Swagger sur /docs. |
| Base de données | PostgreSQL 16 (SQLite en développement) | Données ; migrations versionnées Alembic appliquées au démarrage. |
| Module analytique | Python, scikit-learn | Chaque nuit : prévision hebdomadaire et score de risque ; l'API ne fait que lire les résultats. |
| Sauvegarde | Archive ZIP vérifiable | Quotidienne ; empreintes SHA-256 ; restauration en une transaction. |
| Proxy HTTPS | Caddy (optionnel) | Certificat automatique et en-têtes de sécurité pour une exposition sur Internet. |

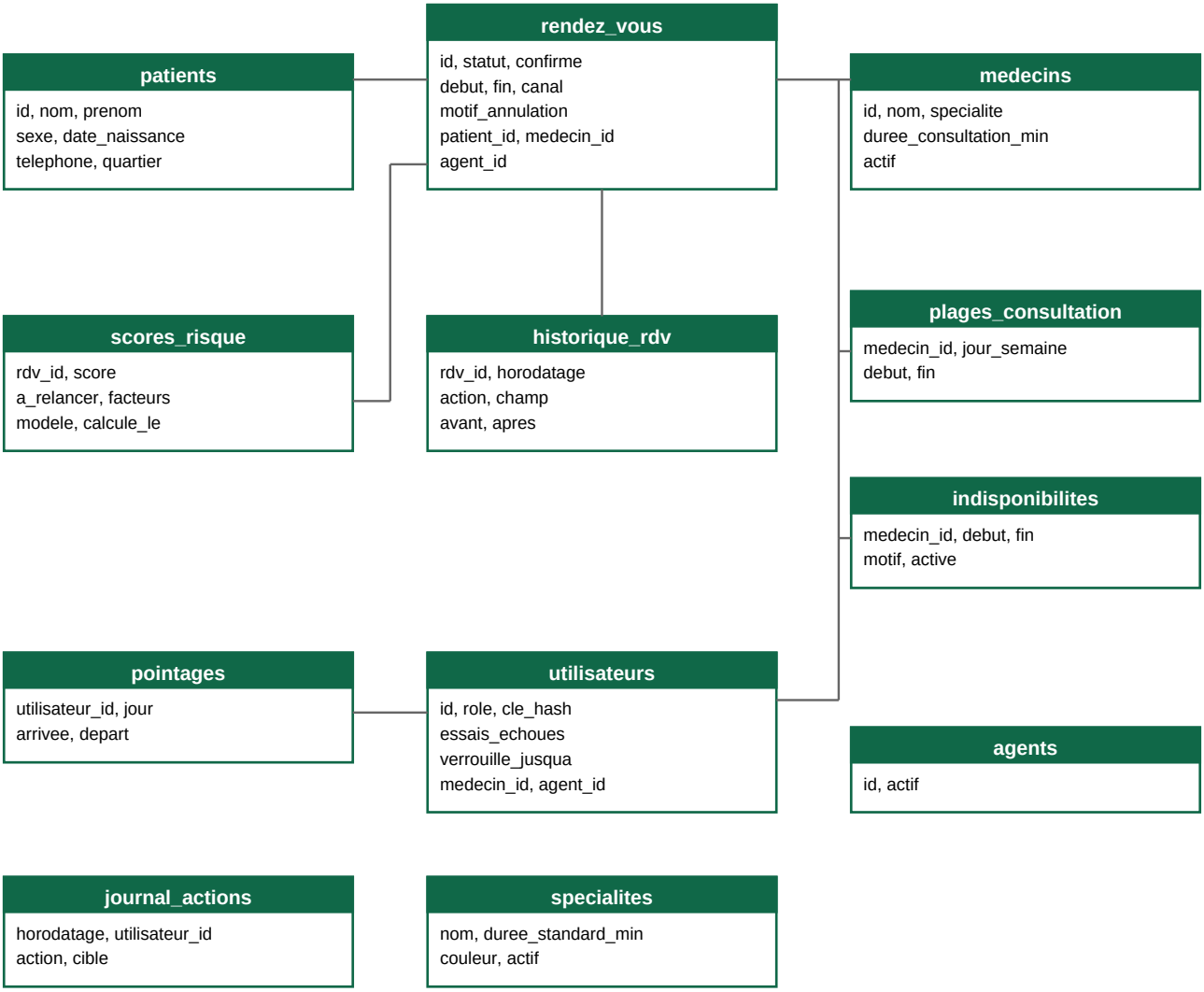
# 3. Choix techniques et justification

| Décision | Raison |
|---|---|
| FastAPI + Pydantic | Validation stricte des entrées, documentation d'API générée, erreurs uniformes {detail, code_erreur}. |
| Logique métier dans l'API | Les règles s'imposent même si l'interface est contournée (cahier des charges, section 6). |
| PostgreSQL en production | Transactions et contraintes d'intégrité ; verrou consultatif pour les migrations concurrentes. |
| Streamlit | Interface rapide à faire évoluer, suffisante pour un usage interne ; adaptation tablette et téléphone native. |
| Authentification par clé personnelle | Clé générée (non choisie), hachée avec Argon2, verrouillage 5 minutes après 5 échecs, session de 8 heures. |
| Modèle de score transparent | Régression logistique comparée à une forêt aléatoire ; seuil de relance déduit de coûts explicites, donc discutable par la clinique. |
| Calcul analytique hors de l'API | Les temps de réponse de l'API ne dépendent pas des calculs lourds. |

# 4. Modèle de données

Le modèle conserve toutes les entités du cahier des charges (section 7.1) et les enrichit : comptes, pointages, journal, historique de rendez-vous, résultats analytiques.

*Chaque trait relie une table à la table qu'elle référence (clé étrangère).*
*La table agents, référencée par rendez_vous et utilisateurs, n'est pas reliée pour garder le schéma lisible.*



| Table | Contenu | Remarque |
|---|---|---|
| patients | Fiche patient (identité, téléphone, quartier). | Anonymisation possible après 5 ans sans rendez-vous. |
| medecins, specialites | Médecins et durée de consultation (15, 20 ou 30 min). | Désactivation plutôt que suppression. |
| plages_consultation | Jours et plages horaires de chaque médecin. | Les créneaux sont calculés à partir des plages. |
| indisponibilites, jours_feries | Absences de médecin, fermetures. | Bloquent la prise de rendez-vous (RG-03). |
| rendez_vous | Rendez-vous et statut (planifié, confirmé, honoré, absent, annulé). | Aucune suppression physique (RG-11). |
| historique_rdv | Chaque modification : qui, quand, avant, après. | Historisation (RG-11). |
| agents, utilisateurs | Comptes de connexion et rôles. | Clés stockées sous forme hachée uniquement. |
| pointages | Arrivée et départ du personnel. | Purgés après 24 mois. |
| journal_actions | Actions sensibles. | Identifiants uniquement, jamais de nom de patient ; purgé après 24 mois. |
| scores_risque, evaluations_risque | Score par rendez-vous à venir et qualité du modèle. | Écrits par le module nocturne ; évaluations purgées après 12 mois. |

# 5. Règles de gestion : où et comment elles sont garanties

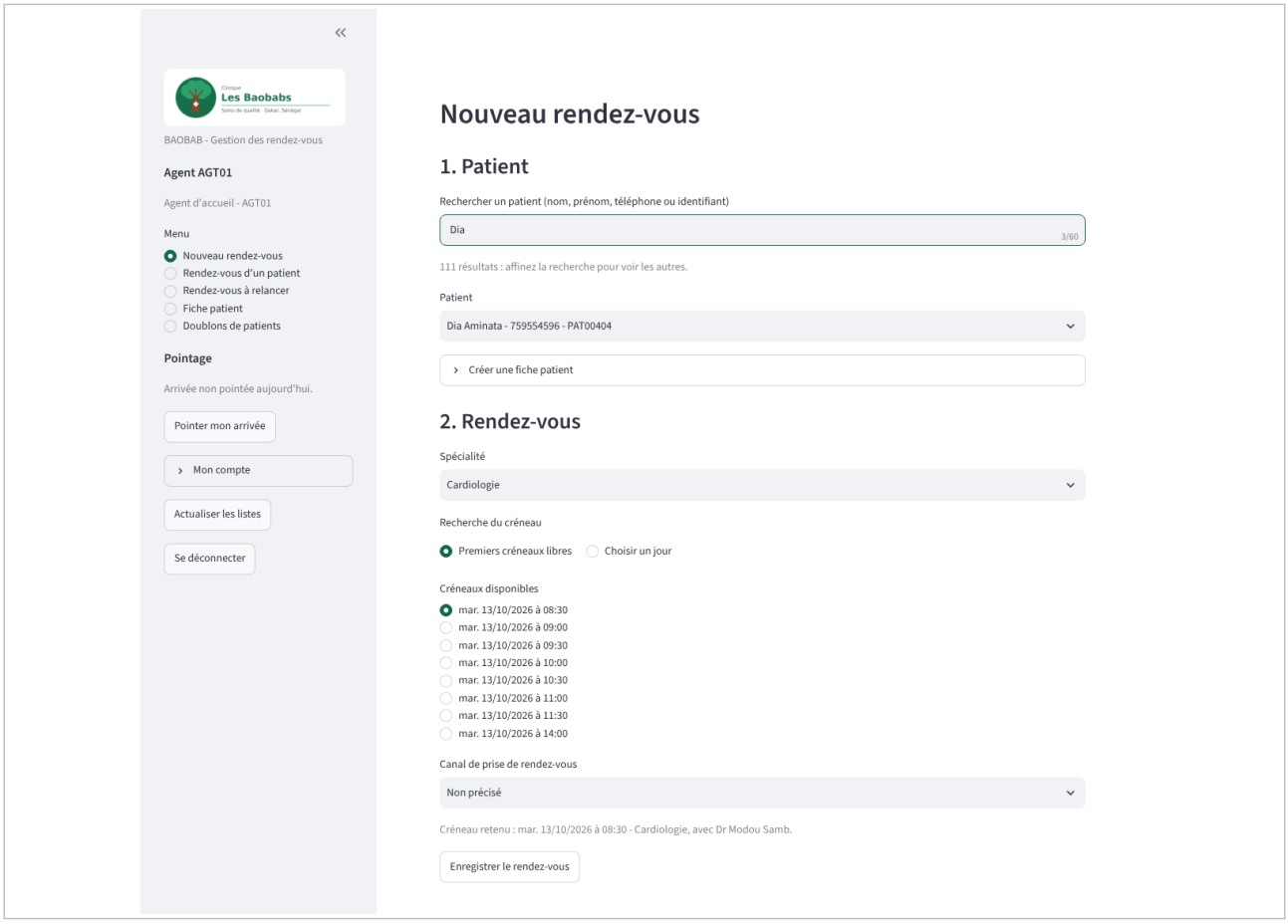
| Règle | Garantie | Tests |
|---|---|---|
| RG-01 un seul rendez-vous actif par créneau | Contrôle serveur à la création et au report ; conflit renvoyé avec message. | test_rdv_regles.py |
| RG-02 pas de chevauchement pour un patient | Contrôle serveur, toutes spécialités confondues ; aussi pris en compte dans la recherche de créneau. | test_rdv_regles.py, test_planning.py |
| RG-03 plages ouvertes, hors fériés et indisponibilités | Créneaux calculés par le serveur ; refus hors plage. | test_planning.py |
| RG-04 ni passé ni plus de 90 jours | Contrôle serveur. | test_rdv_regles.py |
| RG-05 motif d'annulation obligatoire | Liste fermée de motifs, refus sans motif. | test_rdv_regles.py |
| RG-06 annulation libère le créneau | Seuls les statuts actifs occupent un créneau. | test_rdv_regles.py, test_planning.py |
| RG-07 qui peut marquer honoré ou absent | Droits vérifiés côté serveur par rôle et par médecin. | test_rdv_regles.py |
| RG-08 alerte à 3 absences en 6 mois | Avertissement non bloquant renvoyé avec la création. | test_rdv_regles.py |
| RG-09 taux d'absence | absents / (honorés + absents), annulations exclues. | test_stats.py |
| RG-10 taux d'occupation | créneaux honorés ou absents / créneaux ouverts ; recalculé par requêtes indépendantes dans les tests. | test_stats.py, test_planning.py |
| RG-11 historique, pas de suppression | Table d'historique ; aucune route de suppression de rendez-vous. | test_planning.py |

# 6. Sécurité

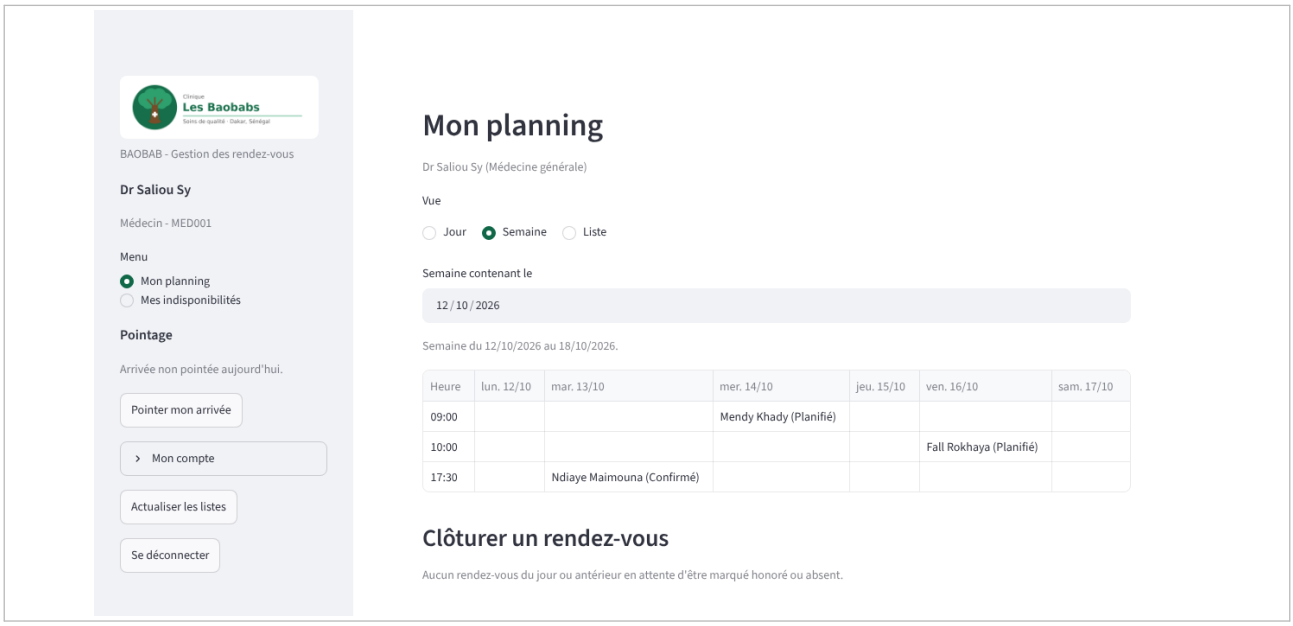
- **Authentification** : clés hachées Argon2, jeton signé, verrouillage après 5 échecs (5 minutes), réponse identique pour un identifiant inconnu ou une mauvaise clé.
- **Autorisation** : le rôle vient du compte, jamais du client ; chaque route déclare les rôles admis (réponse 403 sinon).
- **Entrées** : validation Pydantic, requêtes paramétrées (pas de SQL construit à la main) ; protection contre l'injection de formules dans les exports CSV.
- **En-têtes HTTP** : nosniff, anti-affichage en cadre, politique de référent, cache désactivé ; HSTS derrière le proxy HTTPS.
- **Journaux** : horodatés ; les valeurs des paramètres de requête sont masquées pour ne laisser aucun nom ni numéro de téléphone.
- **Secrets** : jamais dans le dépôt (fichier .env, clés de démonstration, sauvegardes exclus).
- **Données personnelles** : durées de conservation définies (5 ans pour les patients, 24 mois pour journal et pointages) avec purge et anonymisation prévues.

# 7. Maquettes (captures de l'application)

Les écrans suivants sont des captures de l'application réelle sur des données de démonstration synthétiques.




*Prise de rendez-vous : patient, spécialité, créneaux libres.*




*Planning d'un médecin, vue semaine.*

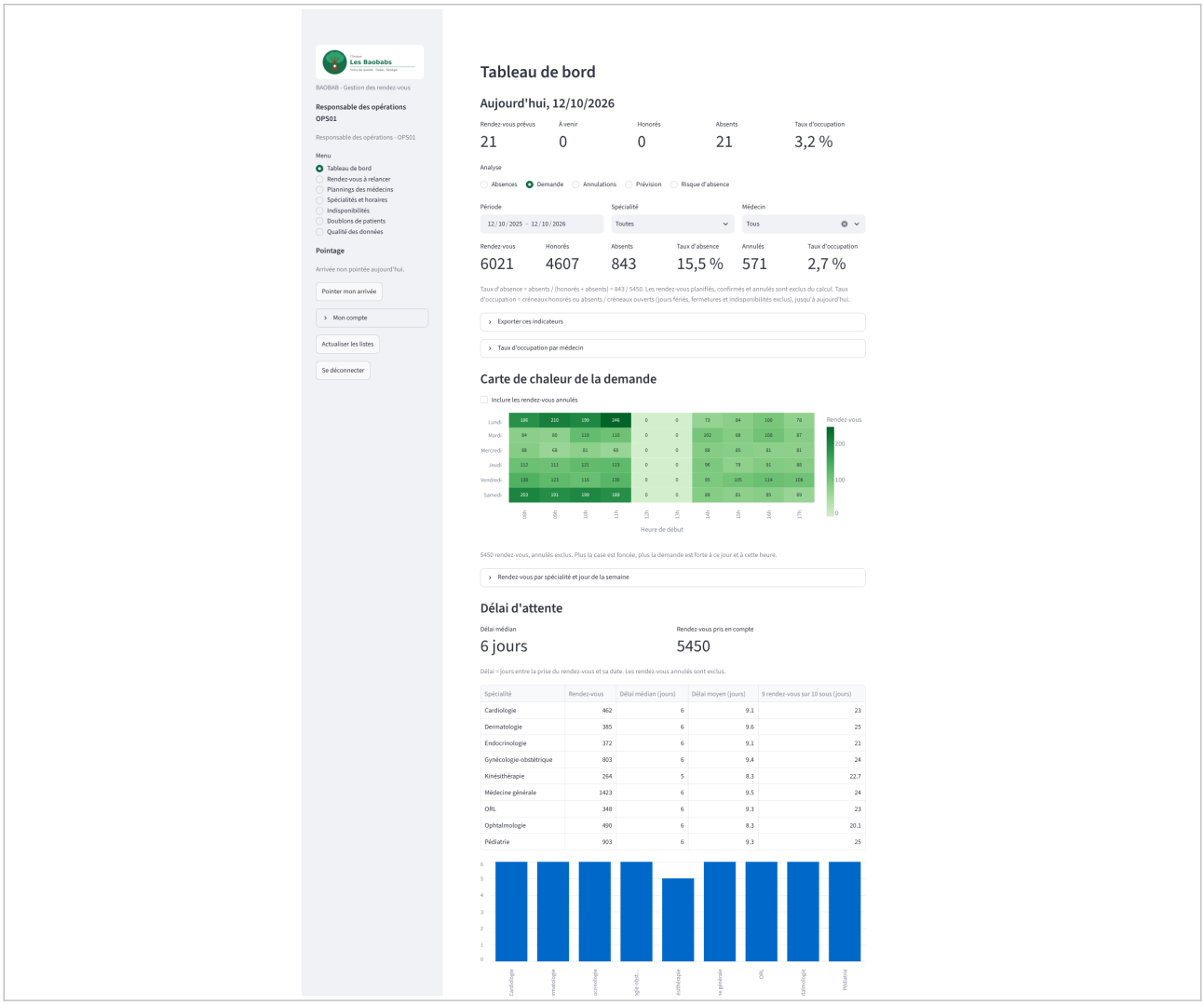



*Tableau de bord : carte de chaleur de la demande.*

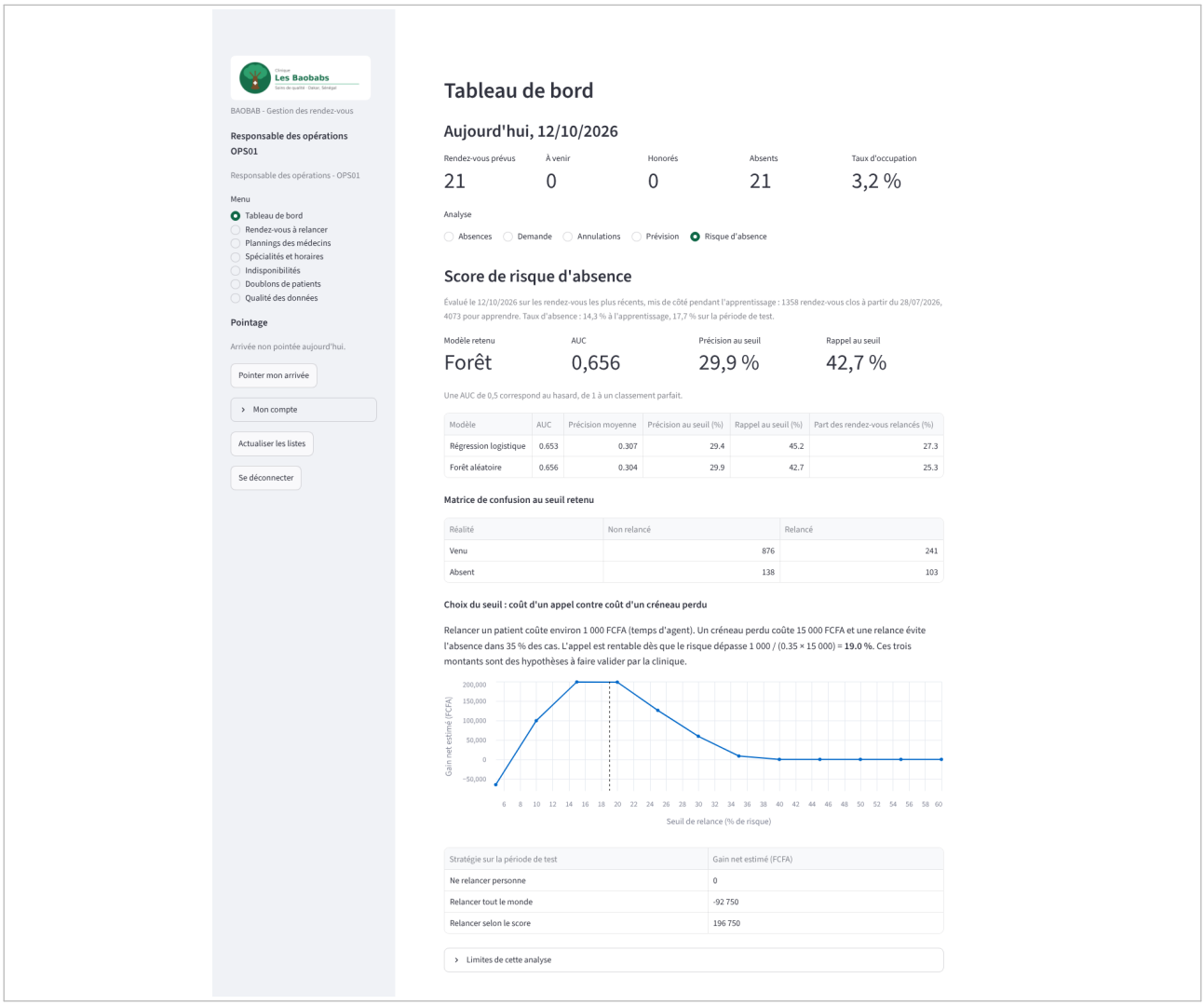
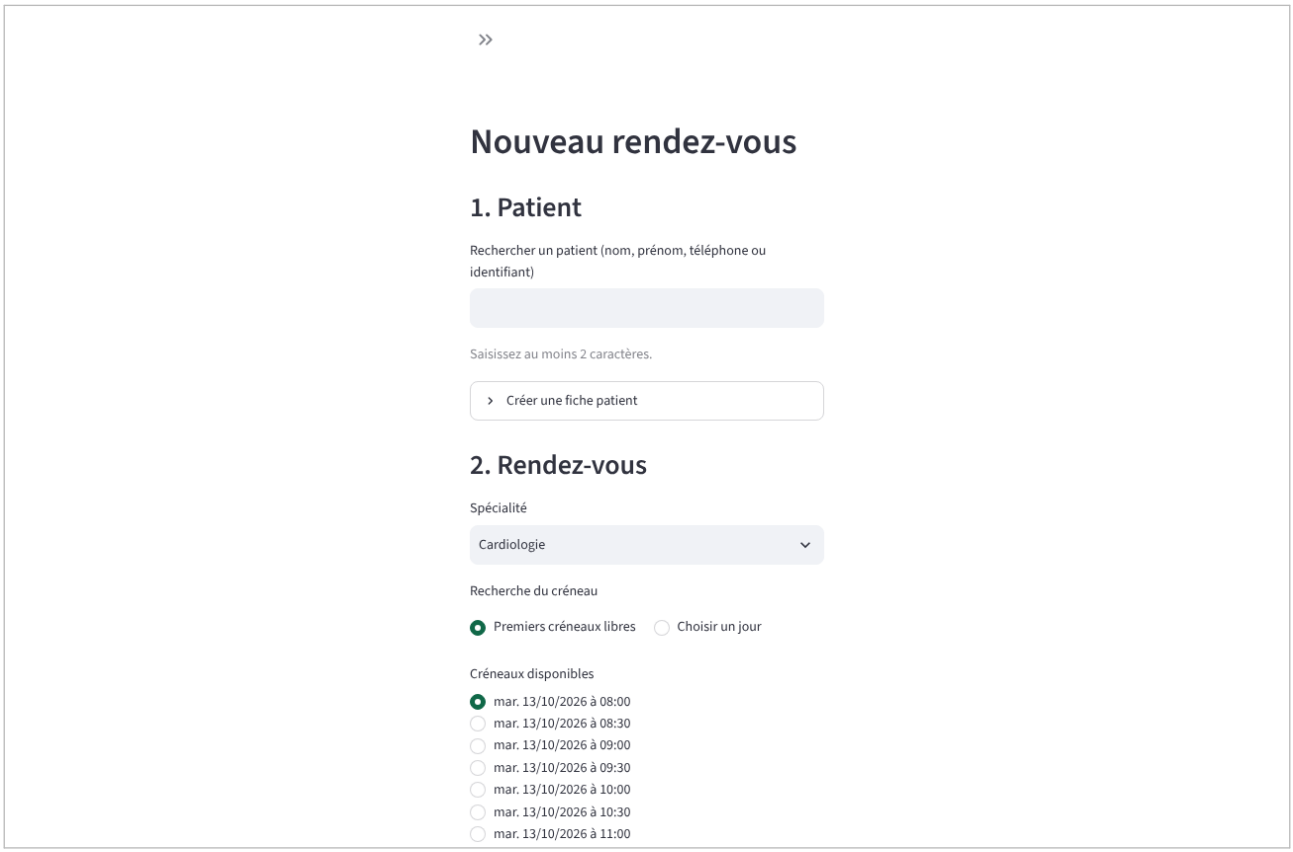



*Tableau de bord : score de risque d'absence.*




*Affichage sur téléphone (390 px de large).*

# 8. Analyse et prévision

**Indicateurs.** Taux d'absence, d'occupation, d'annulation, délai d'attente, carte de chaleur : calculés sur les mêmes définitions que le cahier des charges (RG-09, RG-10) et validés par des requêtes SQL indépendantes.

**Prévision.** Trois méthodes comparées sur un historique caché (moyenne mobile, lissage de Holt, régression calendaire) ; la méthode retenue par spécialité est celle qui se trompe le moins, et l'erreur est affichée.

**Score de risque.** La cible est « absent » contre « honoré » (annulations exclues). Seules des informations connues avant le rendez-vous sont utilisées ; l'évaluation se fait sur les rendez-vous les plus récents (découpage temporel). Le seuil de relance vient d'une formule de coûts : coût d'un appel / (efficacité de la relance x coût d'un créneau perdu), soit 1 000 / (0,35 x 15 000), environ 19 %.

> **Limite à connaître.** Les données de démonstration sont synthétiques : la qualité du score (AUC d'environ 0,65) est modérée et ne préjuge pas des résultats sur les données réelles. Les coûts de relance, de créneau et l'efficacité sont des hypothèses à valider par la clinique ; elles se règlent par variables d'environnement.

# 9. Plan de tests

La suite automatisée compte **221 tests**, tous réussis, avec une couverture de **88 %** du code (rapport HTML fourni, livrable L5). L'objectif du cahier des charges est d'au moins 70 % sur la logique métier.

| Niveau | Contenu | Fichiers |
|---|---|---|
| Unitaire | Règles de calcul, jours fériés, créneaux, modèles analytiques. | test_unitaires.py, test_prevision.py, test_risque.py |
| Intégration API | Chaque route, chaque rôle, cas autorisés et refusés, règles RG-01 à RG-11. | test_rdv_regles.py, test_planning.py, test_patients.py, test_auth.py |
| Sécurité | Accès sans jeton, jeton falsifié, rôle non autorisé, verrouillage, injection, en-têtes. | test_auth.py, test_export_securite.py |
| Validation analytique | Indicateurs recalculés par SQL indépendant et comparés exactement. | test_stats.py, test_analyse.py |
| Exploitation | Sauvegarde, restauration, conservation, journal d'accès. | test_sauvegarde.py, test_conservation.py, test_journal_acces.py |
| Charge | 20 utilisateurs simultanés, tableau de bord sur 12 mois (script dédié, hors pytest). | tests/charge/charge.py |

## Mesures d'exigences non fonctionnelles

| Exigence | Objectif | Mesure | Conditions |
|---|---|---|---|
| ENF-01 | 95 % des requêtes sous 500 ms à 20 utilisateurs | 95e centile : 252 ms | PostgreSQL, 7 500 rendez-vous, poste de développement |
| ENF-02 | Tableau de bord sous 3 s sur 12 mois | 0,93 s seul ; 1,18 s sous charge | idem |
| ENF-11 | Utilisable sur tablette et téléphone | Aucun défilement horizontal à 820 px et 390 px | Mesure par navigateur automatisé |
| ENF-12 | Rendez-vous standard en 5 clics au plus | 4 clics | Patient existant, premier créneau |
| ENF-13 | Navigation au clavier, contrastes | Connexion entièrement au clavier ; couleurs du thème Streamlit | Niveau AA non audité formellement |

Les mesures de charge sont à refaire sur la machine de recette : elles dépendent du matériel.

# 10. Conformité aux exigences

État de chaque exigence au moment de la livraison pour recette. « Livré » signifie implémenté et couvert par au moins un test automatique ou une mesure documentée.

| Exigence | Priorité | État | Commentaire |
|---|---|---|---|
| EF-01 à EF-03 | P1 | Livré | Spécialités, médecins et plages, fiches patients. |
| EF-04 | P2 | Livré (détection) | Doublon signalé à la création ; la fusion de doublons existants n'est pas automatisée. |
| EF-05 | P2 | Livré | Jours fériés calculés (dont fêtes mobiles) et fermetures. |
| EF-10, EF-11 | P1 | Livré | Créneaux générés ; premiers créneaux libres. |
| EF-12 à EF-14 | P1 | Livré | Création, report, annulation avec motif ; anti double réservation ; indisponibilités avec liste à reprogrammer. |
| EF-15 | P1 | Livré | Vues Jour, Semaine et Liste ; filtres par spécialité et médecin. |
| EF-16, EF-17 | P1, P2 | Livré | Statuts et absence automatique une heure après le début. |
| EF-18 | P3 | Non livré | Liste d'attente : optionnelle, non réalisée. |
| EF-20 à EF-23 | P1 | Livré | Indicateurs du jour, carte de chaleur, évolution mensuelle, filtres. |
| EF-24, EF-25 | P2 | Livré | Prévision à 4 semaines ; score de risque et liste à relancer. |

| Exigence | Priorité | État | Commentaire |
|---|---|---|---|
| EF-26 | P2 | Livré | Export CSV et PDF. |
| EF-30, EF-31 | P1 | Livré avec écart | Authentification par identifiant et clé personnelle générée (au lieu d'un mot de passe libre), pour de meilleures garanties. |
| EF-32 | P1 | Livré | Journal des actions. |
| EF-33 | P3 | Non livré | Simulation des rappels : optionnelle, non réalisée. |
| RG-01 à RG-11 | P1 | Livré | Contrôlées côté serveur ; voir section 5. |
| ENF-01, ENF-02 | P1 | Livré | Mesures section 9, à confirmer sur la machine de recette. |
| ENF-06, ENF-07 | - | Livré | En-têtes de sécurité ; HTTPS optionnel (non éprouvé sur un domaine réel à ce jour). |
| ENF-08, ENF-10 | P2 | Livré | Conservation et sauvegarde quotidienne avec restauration testée. |
| ENF-11, ENF-12 | P1, P2 | Livré | Mesures section 9. |
| ENF-13 | P3 | Partiel | Clavier vérifié ; audit de contraste AA non réalisé. |
| ENF-14 | P1 | Livré | Couverture 88 % ; code commenté en français, vérifié avec pyflakes. |
| ENF-15 | P1 | Livré | Une seule commande Docker Compose. |
| ENF-16 | P2 | Livré | Journaux horodatés, paramètres de requête masqués. |

# 11. Déploiement, exploitation et risques

- **Déploiement** : copier .env.example en .env, puis « docker compose up -d --build ». Au premier lancement, les migrations, les données de démonstration et les comptes sont créés ; le fichier de clés produit doit être distribué puis supprimé.
- **Exploitation** : sauvegarde quotidienne (14 copies), vérification et restauration documentées ; purge de conservation à lancer chaque mois (simulation par défaut).
- **Risque : calcul analytique sur données réelles.** Les performances du score doivent être réévaluées dès que l'historique réel est chargé.
- **Risque : une seule instance de l'API.** Suffisant pour la charge demandée ; plusieurs processus possibles sans changer le code.
- **Risque : HTTPS.** Configuration fournie mais à valider sur le nom de domaine réel de la clinique.
- **Évolutions possibles** : liste d'attente, rappels automatiques, fusion des doublons, portail patient.